

Дизайн-документ (SDD): Платформа для создания миров (Worldbuilding Tool)

1. Концепция

Приложение-инструмент для **Мастеров НРИ** и **писателей**. Позволяет создавать структуру вымышленного мира, визуализировать географию, хронологию и связи между сущностями.

- **Приватность:** Все данные приватны по умолчанию.
 - **Изоляция:** Нет социальных функций (лайков, чатов, лент).
 - **Суть:** набор связанных Markdown-страниц и интерактивных визуализаций.
-

2. Функциональные требования

2.1. Пользовательские функции

1. **Регистрация и Авторизация:**
 - Вход по Email/Паролю и Google Account.
 - Восстановление доступа.
2. **Система Ролей (RBAC):**
 - **Admin:** Полный доступ к настройкам сервера/приложения.
 - **Owner (Владелец мира):** Создатель мира, полный контроль над контентом.
 - **Editor:** Может править текст и карты, но не может удалять мир.
 - **Viewer:** Только просмотр.
 - *Кастомные роли:* Настройка прав доступа к конкретным страницам или картам.
3. **Markdown-страницы:**
 - Создание и редактирование статей.
 - Поддержка синтаксиса Markdown.
 - Вставка изображений (загрузка на сервер).
4. **Интерактивная карта (Flat Map):**
 - Загрузка изображения карты (как картинки-подложки).
 - Координатная система: Простая X/Y (пиксельные координаты), без географической привязки (без GPS/LatLon).
 - Добавление маркеров (точек интереса) с описанием.
5. **Таймлапс (Слайдер времени):**
 - Фильтрация маркеров на карте в зависимости от выбранного года/эпохи на слайдере.

6. **Диаграмма отношений (Граф):**
 - Визуализация связей (Node-Edge) между статьями/персонажами.
7. **Таймлайны:**
 - Линейный список событий с датами.
8. **Поиск:**
 - Глобальный поиск по заголовкам и содержанию страниц, названиям маркеров.

2.2. Архитектура и Масштабируемость

1. **Горизонтальная масштабируемость:** Балансировка трафика между несколькими копиями приложения.
2. **Stateless Backend:** Сессии хранятся во внешнем хранилище (Redis), сервер не помнит пользователя "в лицо".
3. **Хранение файлов:** Все изображения (карты, иллюстрации) лежат в объектном хранилище (MinIO), а не в папке сервера.

3. Стек разработки

Frontend

- **Фреймворк:** React + Vite.
- **Стейт-менеджмент:** Zustand.
- **UI библиотека:** Mantine.
- **Карты:** React-Leaflet (используем режим CRS.Simple для работы с картинками без географии).
- **Графы:** React Flow.
- **Валидация форм:** Zod.

Backend

- **Платформа:** Node.js.
- **Фреймворк:** Express.js (быстрый старт, гибкость, минимум шаблонов).
- **ORM:** Prisma (лучшая типизация, простая схема данных).
- **Валидация:** Zod (валидация входящих запросов).

Данные и Инфраструктура

- **База данных:** PostgreSQL (хранение текстов, связей, координат X/Y). PostGIS не нужен.
- **Кэш и Сессии:** Redis.
- **Файловое хранилище:** MinIO (Docker-контейнер, имитирующий AWS S3).
- **Балансировщик:** Nginx.
- **Контейнеры:** Docker Compose.

4. План разработки (Итерации)

Итерация 1: Фундамент (Core)

- Инициализация репозитория (React + Express).
- Настройка **Docker Compose** (Postgres + Redis + MinIO).
- Подключение **Prisma** к БД.
- Реализация Auth (Passport.js или JWT + Redis) и ролей.

Итерация 2: Контент (Markdown & Files)

- CRUD для страниц (Create, Read, Update, Delete).
- Настройка **MinIO**: загрузка аватарок и иллюстраций для статей.
- Интеграция Markdown-редактора и просмотрщика на фронтенде.

Итерация 3: Картография (Flat Maps)

- Загрузка "тяжелых" изображений карт в MinIO.
- Настройка React-Leaflet в режиме CRS.Simple (плоская карта).
- Реализация CRUD маркеров (координаты X, Y храним как Float в Postgres).
- Логика слайдера времени (фильтрация: marker.year <= current_slider_value).

Итерация 4: Связи и Графы

- Реализация диаграммы связей через React Flow.
- Интеграция глобального поиска.
- Доработка системы прав (шеринг конкретных страниц).

Итерация 5: Масштабирование (Scale)

- Настройка **Nginx** конфига для балансировки (upstream).
- Дублирование контейнеров API в Docker Compose.
- Проверка работы сессий через Redis при перезагрузке одного из серверов.

5. Требуемое время (Оценка с ИИ)

Этап	Задачи	Оценка (часы)
------	--------	---------------

1. Фундамент	Setup, Docker, Auth, Prisma Schema	15 - 20
2. Контент	Markdown, File Upload (MinIO)	12 - 16
3. Карты	Leaflet (Simple CRS), Time Logic	20 - 30
4. Связи/Роли	Graph Logic, Search, Sharing	20 - 25
5. Scale	Nginx, Docker scaling, Testing	10 - 15
Отладка	UI Polish, Bugfix	15 - 20
ИТОГО		~90 - 125 часов